

**LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 1
MODUL 17
“SKEMA PEMROSESAN SEKUENSIAL”**



DISUSUN OLEH:
Rayhan Ahza Widyamukti
109082500210
S1 IF-13-02
DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng

**PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026**

DASAR TEORI

Pemrograman merupakan proses penyusunan langkah-langkah logis untuk menyelesaikan suatu permasalahan dengan bantuan komputer. Dalam penyelesaiannya, program dibangun menggunakan algoritma yang tersusun secara sistematis sehingga data masukan dapat diolah menjadi informasi yang dibutuhkan.

Pemrosesan sekuensial adalah metode pengolahan data yang dilakukan secara berurutan sesuai urutan masuknya data. Setiap data diproses satu per satu hingga seluruh data selesai diolah. Metode ini sering digunakan pada perhitungan yang melibatkan banyak data karena lebih sederhana dan efisien dalam penggunaan memori.

Perulangan merupakan struktur kontrol yang memungkinkan suatu instruksi dijalankan berulang kali. Dengan menggunakan perulangan, program dapat memproses sejumlah data tanpa harus menuliskan perintah yang sama secara berulang. Perulangan banyak digunakan untuk melakukan perhitungan, pencarian, simulasi, dan pengolahan kumpulan data.

Bilangan acak adalah bilangan yang dihasilkan oleh komputer untuk meniru sifat acak yang terdapat pada kejadian nyata. Dalam pemrograman, bilangan acak sering digunakan untuk simulasi berbagai fenomena yang sulit diprediksi secara pasti, seperti penyebaran titik, percobaan statistik, maupun pemodelan kejadian alam.

Simulasi merupakan teknik untuk meniru suatu kondisi atau proses nyata ke dalam bentuk model komputasi. Melalui simulasi, suatu permasalahan dapat dianalisis tanpa harus melakukan percobaan secara langsung. Hasil simulasi biasanya akan semakin mendekati kondisi sebenarnya jika jumlah data atau percobaan yang digunakan semakin banyak.

Pada praktikum ini digunakan konsep simulasi untuk memperkirakan nilai tertentu dan menghitung distribusi data pada suatu bidang. Pendekatan tersebut memanfaatkan banyak percobaan berulang dan pengolahan data secara sekuensial. Dengan menggabungkan perulangan, bilangan acak, dan proses perhitungan bertahap, program dapat menghasilkan estimasi yang semakin akurat seiring bertambahnya jumlah data yang diproses.

SOAL LATIHAN

Statement perulangan

1. Mencari Rerata

Source Code:

```
package main

import "fmt"

func main() {
    var bilangan float64
    var jumlah float64
    var rataRata float64
    var banyak int

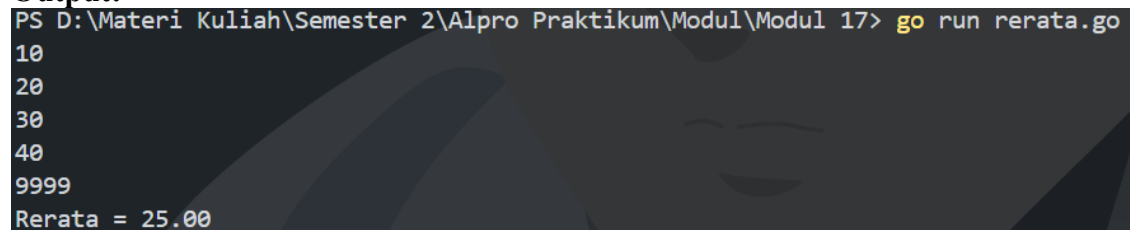
    jumlah = 0
    banyak = 0

    fmt.Scan(&bilangan)

    for bilangan != 9999 {
        jumlah = jumlah + bilangan
        banyak = banyak + 1
        fmt.Scan(&bilangan)
    }

    if banyak > 0 {
        rataRata = jumlah / float64(banyak)
        fmt.Printf("Rerata = %.2f\n", rataRata)
    } else {
        fmt.Println("Tidak ada data yang dihitung")
    }
}
```

Output:



```
PS D:\Materi Kuliah\Semester 2\Alpro Praktikum\Modul\Modul 17> go run rerata.go
10
20
30
40
9999
Rerata = 25.00
```

Deskripsi Program:

Program membaca bilangan real satu per satu menggunakan `fmt.Scan()`. Selama nilai yang dimasukkan bukan 9999, nilai tersebut ditambahkan ke variabel `jumlah` dan

penghitung data banyak ditambah satu. Ketika pengguna memasukkan 9999, proses pembacaan berhenti karena nilai tersebut hanya berfungsi sebagai marker dan tidak ikut dihitung. Setelah itu, rerata diperoleh dengan membagi total jumlah dengan banyak data yang telah dibaca, lalu hasilnya ditampilkan ke layar.

2. Mencari String

Source Code:

```
package main

import "fmt"

func main() {
    var x, data string
    var n, i int
    var ditemukan bool
    var posisi int
    var jumlah int

    fmt.Print("Masukkan string x: ")
    fmt.Scan(&x)

    fmt.Print("Masukkan jumlah data (n): ")
    fmt.Scan(&n)

    ditemukan = false
    posisi = -1
    jumlah = 0

    for i = 1; i <= n; i++ {
        fmt.Print("Data ke-", i, ": ")
        fmt.Scan(&data)

        if data == x {
            jumlah = jumlah + 1
            ditemukan = true

            if posisi == -1 {
                posisi = i
            }
        }
    }

    fmt.Println()
    fmt.Println("Jawaban:")
}
```

```

        // a
        if ditemukan {
            fmt.Println("a. String x ada dalam kumpulan
data.")
        } else {
            fmt.Println("a. String x tidak ada dalam
kumpulan data.")
        }

        // b
        if posisi != -1 {
            fmt.Println("b. String x pertama kali ditemukan
pada posisi", posisi)
        } else {
            fmt.Println("b. String x tidak ditemukan.")
        }

        // c
        fmt.Println("c. Jumlah string x dalam kumpulan data
=", jumlah)

        // d
        if jumlah >= 2 {
            fmt.Println("d. Ya, terdapat sedikitnya dua
string x.")
        } else {
            fmt.Println("d. Tidak, terdapat kurang dari dua
string x.")
        }
    }
}

```

Output:

```

PS D:\Materi Kuliah\Semester 2\Alpro Praktikum\Modul\Modul 17> go run bilangan.go
Masukkan string x: apel
Masukkan jumlah data (n): 5
Data ke-1: mangga
Data ke-2: apel
Data ke-3: anggur
Data ke-4: sirsak
Data ke-5: jeruk

Jawaban:
a. String x ada dalam kumpulan data.
b. String x pertama kali ditemukan pada posisi 2
c. Jumlah string x dalam kumpulan data = 1
d. Tidak, terdapat kurang dari dua string x.

```

Deskripsi Program:

Program ini menggunakan skema pemrosesan sekuensial, yaitu data diproses satu per satu sesuai urutan input. Program menerima sebuah string *x* sebagai data yang dicari, kemudian menerima nilai *n* yang menyatakan banyaknya data string yang akan dibaca. Setiap data yang dimasukkan langsung dibandingkan dengan *x*. Jika sama, program akan mencatat bahwa string ditemukan, menyimpan posisi pertama kemunculannya, dan menambah penghitung jumlah kemunculan. Setelah seluruh data selesai diproses, program menampilkan jawaban untuk empat pertanyaan, yaitu keberadaan string *x*, posisi pertama kemunculannya, jumlah kemunculannya, dan apakah terdapat sedikitnya dua kemunculan string *x* dalam kumpulan data tersebut.

Pertanyaan:**A. Apakah String *x* ada dalam kumpulan *n* data string tersebut?**

Jawaban untuk poin ini diperoleh dari variabel ditemukan. Jika selama proses pembacaan data terdapat string yang sama dengan *x*, maka nilai ditemukan akan menjadi true. Setelah seluruh data selesai dibaca, program dapat menentukan apakah string *x* ada atau tidak dalam kumpulan data.

B. Pada posisi ke berapa string *x* tersebut ditemukan?

Jawaban untuk poin ini diperoleh dari variabel posisi. Saat string *x* ditemukan pertama kali, indeks atau urutan data tersebut disimpan ke dalam variabel posisi. Dengan demikian program dapat menampilkan posisi pertama kemunculan string *x* pada kumpulan data.

C. Ada berapakah string *x* dalam kumpulan *n* data string tersebut?

Jawaban untuk poin ini diperoleh dari variabel jumlah. Setiap kali data yang dibaca sama dengan string *x*, nilai jumlah ditambah satu. Setelah seluruh data diproses, nilai jumlah menunjukkan banyaknya kemunculan string *x* dalam kumpulan data.

D. Adakah sedikitnya dua string dalam *n* data string tersebut?

Jawaban untuk poin ini diperoleh dengan memeriksa nilai jumlah. Jika jumlah bernilai 2 atau lebih, maka dapat disimpulkan bahwa terdapat sedikitnya dua string *x* dalam kumpulan data. Sebaliknya, jika jumlah kurang dari 2, maka syarat tersebut tidak terpenuhi.

3. Daerah Curah Hujan

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
)

func main() {
    var n int
    var i int
    var x, y float64
    var a, b, c, d int

    fmt.Print("Masukkan jumlah tetesan hujan: ")
    fmt.Scan(&n)

    a = 0
    b = 0
    c = 0
    d = 0

    for i = 1; i <= n; i++ {
        x = rand.Float64()
        y = rand.Float64()

        if x < 0.5 && y < 0.5 {
            a = a + 1
        } else if x >= 0.5 && y < 0.5 {
            b = b + 1
        } else if x >= 0.5 && y >= 0.5 {
            c = c + 1
        } else {
            d = d + 1
        }
    }

    fmt.Printf("Curah hujan daerah A: %.4f milimeter\n",
float64(a)*0.0001)
    fmt.Printf("Curah hujan daerah B: %.4f milimeter\n",
float64(b)*0.0001)
    fmt.Printf("Curah hujan daerah C: %.4f milimeter\n",
float64(c)*0.0001)
    fmt.Printf("Curah hujan daerah D: %.4f milimeter\n",
float64(d)*0.0001)
}
```

Output:

```
PS D:\Materi Kuliah\Semester 2\Alpro Praktikum\Modul\Modul 17> go run daerah.go
Masukkan jumlah tetesan hujan: 10000000
Curah hujan daerah A: 249.8764 milimeter
Curah hujan daerah B: 250.2844 milimeter
Curah hujan daerah C: 249.8771 milimeter
Curah hujan daerah D: 249.9621 milimeter
```

Deskripsi Program:

Program menerima input berupa jumlah tetesan air hujan yang akan disimulasikan. Setiap tetesan diwakili oleh sebuah titik acak (x, y) yang dibangkitkan menggunakan fungsi `rand.Float64()`, dengan nilai berada pada rentang 0 sampai 1. Bidang pengamatan dibagi menjadi empat daerah yang sama besar, yaitu A, B, C, dan D, dengan batas pada $x = 0,5$ dan $y = 0,5$. Untuk setiap titik yang dihasilkan, program memeriksa posisi titik tersebut dan menambahkan penghitung pada daerah yang sesuai. Setelah seluruh tetesan diproses, banyaknya tetesan pada masing-masing daerah dikonversi menjadi curah hujan dengan ketentuan bahwa satu tetesan setara dengan 0,0001 milimeter. Hasil akhir berupa curah hujan pada daerah A, B, C, dan D ditampilkan dalam satuan milimeter dengan empat angka di belakang koma.

4. Mencari Rerata

Source Code:

```
package main

import "fmt"

func main() {
    var n int
    var i int
    var suku float64
    var jumlah float64

    fmt.Print("N suku pertama: ")
    fmt.Scan(&n)

    jumlah = 0

    for i = 1; i <= n; i++ {
```



```

        if i%2 == 1 {
            suku = 1.0 / float64(2*i-1)
        } else {
            suku = -1.0 / float64(2*i-1)
        }

        jumlah = jumlah + suku
    }

    fmt.Printf("Hasil PI: %.7f\n", 4*jumlah)

    i = 1
    jumlah = 0

    for {
        if i%2 == 1 {
            suku = 1.0 / float64(2*i-1)
        } else {
            suku = -1.0 / float64(2*i-1)
        }

        jumlah = jumlah + suku

        if (1.0 / float64(2*i+1)) <= 0.00001 {
            break
        }

        i = i + 1
    }

    fmt.Printf("Hasil PI: %.10f\n", 4*jumlah)
    fmt.Println("Pada i ke:", i)
}

```

Output:

```

PS D:\Materi Kuliah\Semester 2\Alpro Praktikum\Modul\Modul 17> go run formula.go
N suku pertama: 100000
Hasil PI: 3.1415917
Hasil PI: 3.1415726536
Pada i ke: 50000

```

Deskripsi Program:

Program menggunakan deret Leibniz untuk menghitung nilai π . Pada bagian pertama, pengguna memasukkan banyaknya suku yang akan dihitung, kemudian program menjumlahkan suku-suku deret hingga suku ke-n dan mengalikannya dengan 4 untuk memperoleh pendekatan nilai π . Pada bagian kedua, program melakukan perhitungan ulang dengan menambahkan suku satu per satu sampai nilai suku

berikutnya tidak lebih dari 0,00001. Karena selisih antara dua jumlah deret berurutan sama dengan nilai suku yang baru ditambahkan, kondisi tersebut dapat digunakan sebagai syarat penghentian. Setelah syarat terpenuhi, program menampilkan pendekatan nilai π yang diperoleh beserta nilai i saat proses berhenti.

5. Banyak Topping

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
)

func main() {
    var banyakTopping int
    var i int
    var x, y float64
    var toppingPizza int
    var pi float64

    fmt.Print("Banyak Topping: ")
    fmt.Scan(&banyakTopping)

    toppingPizza = 0

    for i = 1; i <= banyakTopping; i++ {
        x = rand.Float64()
        y = rand.Float64()

        if (x-0.5)*(x-0.5)+(y-0.5)*(y-0.5) <= 0.25 {
            toppingPizza = toppingPizza + 1
        }
    }

    pi = 4.0 * float64(toppingPizza) /
float64(banyakTopping)

    fmt.Println("Topping pada Pizza:", toppingPizza)
    fmt.Printf("PI : %.10f\n", pi)
}
```

Output:

```
PS D:\Materi Kuliah\Semester 2\Alpro Praktikum\Modul\Modul 17> go run kedai.go
Banyak Topping: 1234567
Topping pada Pizza: 969856
PI : 3.1423357339
```

Deskripsi Program:

Program mensimulasikan penaburan topping secara acak pada sebuah wadah pizza berbentuk persegi dengan ukuran 1×1 . Titik pusat pizza berada pada koordinat (0.5, 0.5) dan memiliki jari-jari 0.5 sehingga pizza berbentuk lingkaran yang tepat berada di dalam wadah persegi. Setiap topping direpresentasikan sebagai sebuah titik acak (x, y) yang dibangkitkan menggunakan fungsi `rand.Float64()`. Sebuah topping dianggap jatuh di atas pizza apabila memenuhi persamaan lingkaran $(x-0.5)^2 + (y-0.5)^2 \leq 0.25$. Program menghitung jumlah topping yang berada di dalam lingkaran dan menyimpannya pada variabel `toppingPizza`.

Nilai π dihitung menggunakan metode Monte Carlo berdasarkan perbandingan luas lingkaran terhadap luas persegi. Karena luas lingkaran dengan jari-jari 0.5 adalah $\pi/4$ dan luas persegi adalah 1, maka perbandingan banyak titik yang jatuh di dalam lingkaran terhadap seluruh titik acak akan mendekati $\pi/4$. Oleh karena itu, nilai π dapat diperkirakan dengan rumus $\pi = 4 \times (\text{jumlah topping pada pizza} / \text{jumlah seluruh topping})$. Semakin banyak topping yang digunakan dalam simulasi, semakin mendekati nilai π yang sebenarnya.

DAFTAR PUSTAKA

- Munir, R. (2018). *Algoritma dan Pemrograman dalam Bahasa Pascal, C, dan C++*. Bandung: Informatika.
- Kurniawan, B. (2020). *Pemrograman Go untuk Pemula*. Jakarta: Elex Media Komputindo.
- Donovan, A. A., & Kernighan, B. W. (2016). *The Go Programming Language*. Boston: Addison-Wesley Professional.
- Gries, D., & Schneider, F. B. (1993). *A Logical Approach to Discrete Math*. New York: Springer.